PLAN DE DESPLIEGUE DEL SOFTWARE

PROYECTO: Traduce Señas

INTEGRANTES:
Luis Alejandro Duarte Aldana
Juan Mauricio Suaza Solorzano
Juan Camilo Fierro Esquivel

SERVICIO NACIONAL DE APRENDIZAJE – SENA
Análisis y Desarrollo De Software

2026

Tabla de contenido

Introducción 4
Información general y control de versiones 4
Ficha técnica del despliegue 4
Identificación del sistema y versión 4
Fecha, hora y ventana operativa programada 4
Entorno de destino 5
Matriz de roles y responsables 5
Historial de versiones del documento 5
Resumen de cambios (Release Notes) 5
Nuevas funcionalidades 5
Correcciones de errores (Bug fixes) 6
Cambios de infraestructura y dependencias 6
Cambios incompatibles (Breaking changes) 6
Alcance, topología y arquitectura 7
Componentes y artefactos en alcance 7
Panel de administración (Portainer CE) 7
Capa frontend web 7
Capa backend API 7
Capa de persistencia 8
Diagrama de arquitectura de red e infraestructura 8
Topología del servidor VPS 8
Red interna de contenedores 8
Enrutamiento y reverse proxy con TLS/HTTPS 9
Matriz de ambientes, direccionamiento IP y puertos 9
Requisitos previos y preparación 9
Checklist de prerrequisitos del sistema operativo y del VPS 9
Recursos de hardware disponibles 9
Estado del motor Docker y socket local 10
Estrategia y ejecución de respaldos 10
Respaldo lógico de base de datos 10
Respaldo de volúmenes persistentes de Docker 10
Respaldo de configuraciones de stacks y variables anteriores 11
Gestión de variables de entorno y secretos 11
Parámetros de conexión a base de datos 11
Claves de firmado de tokens 11
Límites de memoria y runtime 11
Protocolo de comunicación y ventana de mantenimiento 12
Procedimiento de despliegue paso a paso (Runbook) 12

Verificación y acceso a la plataforma base (Portainer CE) 12
Despliegue y migración de base de datos 12
Ejecución de scripts DDL/DML o herramientas de migración 12
Verificación de integridad de esquemas, índices y usuarios 13
Despliegue y orquestación del stack de aplicación 13
Carga del archivo docker-compose.yml en Portainer 13
Inyección de variables de entorno del stack 13
Despliegue de servicios y actualización de imágenes de contenedor 14
Configuración de enrutamiento, dominio y certificados SSL/TLS 14
Plan de verificación, pruebas y evidencias (Smoke tests) 15
Matriz de enlaces y endpoints de verificación de servicios 15
Acceso al panel administrativo 15
Interfaz web pública 15
Endpoints de salud del backend 15
Documentación interactiva de la API 15
Conectividad TCP directa a motor de base de datos 16
Pruebas de humo funcionales 16
Flujo de autenticación y emisión de tokens 16
Pruebas de lectura y consulta de datos 16
Pruebas de escritura y persistencia transaccional 16
Inspección de logs y monitoreo de recursos 17
Revisión de trazas de inicio sin excepciones críticas 17
Monitoreo de consumo de CPU y memoria RAM en contenedores 17
Registro de evidencias visuales 17
Captura del panel de Portainer con contenedores en estado running 17
Captura del visor de logs de los servicios backend 17
Captura de respuesta HTTP 200 de los health checks 18
Captura de interfaz web React con consola DevTools limpia 18
Captura de registro de migración exitosa en base de datos 18
Plan de reversión 18
Conclusiones 19
Referencias 19

Introducción

Este plan reúne lo necesario para publicar Traduce Señas en el servidor del proyecto y comprobar que quedó funcionando: quién participa, qué cambia en esta versión, cómo está armada la infraestructura, qué se respalda antes de empezar, los pasos del despliegue y las pruebas con las que se acepta. Complementa el procedimiento de implementación general, que explica la instalación desde cero; aquí el foco es la puesta en producción sobre el VPS.

Información general y control de versiones

Ficha técnica del despliegue

Identificación del sistema y versión

Tabla 1
Identificación del sistema

Campo	Valor
Sistema	Traduce Señas: traductor de Lengua de Señas Colombiana (LSC) a texto
Versión (Release Tag)	v1.0.0
Repositorio backend	github.com/mauro-1508/backendTS, rama develop, commit 028d4ab
Repositorio frontend	github.com/tadeo77789/frontendTS, rama feat/vps-api-url, commit b4f5ef4
Repositorio de documentación	github.com/code-sena/trans-sl-docs, rama main
Tipo de despliegue	Contenedores Docker orquestados con Docker Compose y administrados desde Portainer CE

Nota. La etiqueta v1.0.0 coincide con la versión declarada en el package.json de cada servicio. Se crea en Git al cerrar la ventana de despliegue con «git tag -a v1.0.0» sobre los commits indicados.

Fecha, hora y ventana operativa programada

Tabla 2
Ventana operativa

Actividad	Fecha y hora (Colombia, UTC-5)	Duración
Aviso a usuarios y congelamiento de cambios	15/10/2026, 12:00	—
Respaldos previos	15/10/2026, 18:00	20 min
Despliegue de base de datos y servicios	15/10/2026, 18:20	40 min
Pruebas de humo y registro de evidencias	15/10/2026, 19:00	40 min
Cierre de ventana o reversión	15/10/2026, 19:40	20 min

Se eligió la franja de la tarde porque el uso de la aplicación es bajo y el equipo puede estar conectado al mismo tiempo. Si la ventana se extiende más de 30 minutos sobre lo previsto, se aplica el plan de reversión.

Entorno de destino

Tabla 3
Entornos del proyecto

Entorno	Ubicación	Uso
Local	Equipo de cada integrante con Docker Desktop	Desarrollo y pruebas unitarias
Staging	No disponible todavía	Se reemplaza por una prueba completa con Docker en local antes de cada despliegue
Producción	VPS de Hostinger, IP pública 179.236.77.215	Backend de la aplicación móvil y web

Matriz de roles y responsables

Tabla 4
Roles del despliegue

Rol	Responsable	Tareas
DevOps	Juan Mauricio Suaza Solorzano	Acceso al VPS, Portainer, stack de contenedores, respaldos y reversión
Tech Lead	Juan Mauricio Suaza Solorzano	Aprueba la versión, decide si se continúa o se revierte
DBA	Juan Mauricio Suaza Solorzano	Respaldos de PostgreSQL y MongoDB, verificación de esquemas
Frontend	Luis Alejandro Duarte Aldana	URL del gateway en la aplicación, generación del APK y versión web
QA	Juan Camilo Fierro Esquivel	Pruebas de humo, capturas de evidencia y registro de defectos

Historial de versiones del documento

Tabla 5
Control de cambios del documento

Versión	Fecha	Autor	Descripción
1.0	15/10/2026	Juan Mauricio Suaza Solorzano	Versión inicial del plan para el despliegue en el VPS

Resumen de cambios (Release Notes)

Nuevas funcionalidades

- Backend separado en cinco microservicios (identidad, reconocimiento, léxico, analítica y perfil) detrás de un API Gateway único.
- Eventos entre servicios por RabbitMQ: la primera traducción de un usuario activa el logro «Primera traducción» y queda registrada en las estadísticas.
- Modelos 3D, miniaturas y descripciones del alfabeto actualizados en el servicio de

léxico.

- Inicio de sesión con Facebook en la aplicación y perfiles de compilación para generar el APK de Android.
- La aplicación apunta al gateway publicado en el VPS en lugar de la máquina local.

Correcciones de errores (Bug fixes)

- El servicio de identidad registra al arrancar si el envío de correos está mal configurado, en vez de fallar en silencio al recuperar la contraseña.
- El progreso de los logros se procesa en orden por usuario, para no perder incrementos cuando llegan dos traducciones a la vez.
- Se validan los eventos de traducción antes de procesarlos y la notificación se marca como enviada solo cuando se crea.
- El perfil conserva el nombre del usuario que llega en el evento de registro.
- Cambiar las preferencias de notificación exige el permiso correspondiente.

Cambios de infraestructura y dependencias

- Una base de datos PostgreSQL 16 por servicio, MongoDB 7 para las muestras de entrenamiento y RabbitMQ 3 con panel de administración.
- Solo el gateway (puerto 8080) queda publicado; bases de datos y panel de RabbitMQ se enlazan únicamente a 127.0.0.1.
- Secretos obligatorios: el stack no arranca sin JWT_SECRET de 32 caracteres o más ni sin las contraseñas de las bases y del broker.
- Imágenes de Node.js 20 en Alpine construidas en dos etapas.

Cambios incompatibles (Breaking changes)

- La API ya no se atiende en el puerto 3000: todas las peticiones pasan por el gateway en el 8080 bajo /api. Versiones anteriores de la aplicación dejan de conectarse.

- Los tokens emitidos antes del cambio dejan de ser válidos porque ahora se firman con emisor y audiencia; los usuarios deben iniciar sesión de nuevo.
- CORS funciona con lista blanca: un origen web que no esté en CORS_ORIGINS recibe rechazo.

Alcance, topología y arquitectura

Componentes y artefactos en alcance

Panel de administración (Portainer CE)

Portainer CE se usa para ver el estado de los contenedores, revisar registros y redesplegar el stack sin escribir comandos largos. Corre como un contenedor más, con acceso al socket de Docker, y su puerto 9443 no se publica a internet: se entra mediante un túnel SSH. Con esto el panel queda protegido aunque alguien conozca la IP del servidor.

Capa frontend web

La aplicación está hecha con Expo y React Native, de modo que un mismo código genera la versión móvil (APK) y la versión web. La web se compila a archivos estáticos y se sirve con Nginx, que además redirige las rutas /api hacia el gateway. En esta versión el canal principal es el APK; la versión web se publica en el mismo VPS cuando el dominio con HTTPS esté listo.

Capa backend API

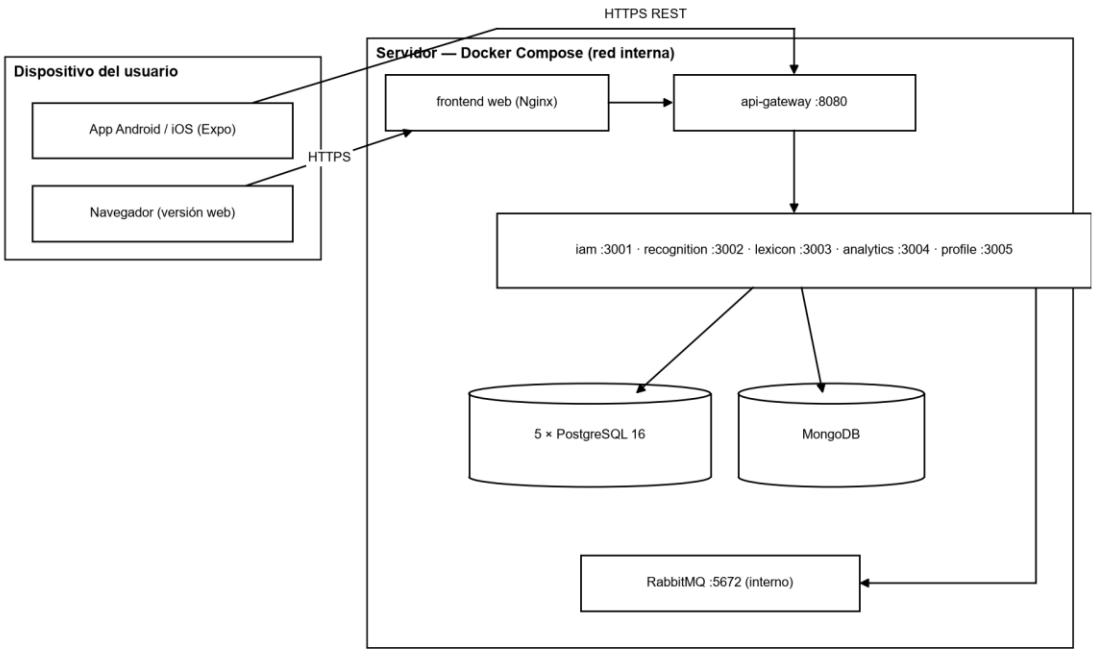
El backend está escrito en TypeScript sobre Node.js 20 con Express. Lo forman el API Gateway, que valida el token, limita peticiones y enruta, y cinco servicios: iam-service (cuentas, roles y recuperación de contraseña), recognition-service (traducciones y plantillas de señas), lexicon-service (catálogo y alfabeto en 3D), analytics-service (eventos de uso) y profile-service (perfil, notificaciones y logros). No se usa Java ni .NET.

Capa de persistencia

Cada servicio es dueño de su base PostgreSQL; ninguno lee la base de otro. El servicio de reconocimiento guarda además las muestras de entrenamiento en MongoDB. Los datos viven en volúmenes de Docker con nombre, por lo que sobreviven a la recreación de los contenedores.

Diagrama de arquitectura de red e infraestructura

Figura 1
Diagrama de despliegue del sistema



Nota. Elaboración propia.

Topología del servidor VPS

El servidor es un VPS Linux de Hostinger con Docker Engine y Docker Compose. Desde internet solo se alcanzan el puerto 22 (SSH, con llave) y el 8080 (gateway). Todo lo demás está cerrado por el firewall o enlazado a la interfaz local.

Red interna de contenedores

Docker Compose crea una red bridge para el stack. Los contenedores se encuentran

por nombre (por ejemplo, el gateway llama a http://iam-service:3001) y los puertos 3001 a 3005, 5432, 27017 y 5672 nunca salen de esa red.

Enrutamiento y reverse proxy con TLS/HTTPS

Hoy la aplicación móvil se conecta por HTTP directo a 179.236.77.215:8080, lo que obligó a permitir tráfico sin cifrar en Android. Este es el principal pendiente de seguridad: el siguiente despliegue pone Nginx como proxy inverso en los puertos 80 y 443 con un certificado de Let's Encrypt para un dominio propio, cierra el 8080 al exterior y cambia la URL de la aplicación a https.

Matriz de ambientes, direccionamiento IP y puertos

Tabla 6
Puertos por componente en producción

Componente	Puerto interno	Publicación en el VPS	Acceso
api-gateway	8080	0.0.0.0:8080	Internet
iam-service	3001	No se publica	Red Docker
recognition-service	3002	No se publica	Red Docker
lexicon-service	3003	No se publica	Red Docker
analytics-service	3004	No se publica	Red Docker
profile-service	3005	No se publica	Red Docker
PostgreSQL (5 instancias)	5432	127.0.0.1:5433 a 5437	Solo desde el VPS o túnel SSH
MongoDB	27017	127.0.0.1:27017	Solo desde el VPS o túnel SSH
RabbitMQ (AMQP / panel)	5672 / 15672	No se publica / 127.0.0.1:15672	Red Docker / túnel SSH
Portainer CE	9443	127.0.0.1:9443	Túnel SSH
SSH	22	0.0.0.0:22	Integrantes con llave autorizada

Requisitos previos y preparación

Checklist de prerrequisitos del sistema operativo y del VPS

Recursos de hardware disponibles

Tabla 7
Recursos mínimos y forma de comprobarlos

Recurso	Mínimo para el stack	Comprobación
CPU	2 vCPU	nproc
Memoria RAM	4 GB (8 GB recomendados)	free -h

Recurso	Mínimo para el stack	Comprobación
Disco NVMe	20 GB libres	df -h /
Sistema operativo	Ubuntu 22.04 LTS o superior	lsb_release -a

Nota. Doce contenedores en reposo consumen cerca de 2 GB de RAM; el resto queda para picos de reconocimiento y para construir imágenes.

Estado del motor Docker y socket local

```
docker version
docker compose version
systemctl is-active docker
ls -l /var/run/docker.sock
```

Docker debe responder «active» y el usuario de despliegue debe pertenecer al grupo docker. Si «docker ps» pide sudo, el usuario no está en el grupo.

Estrategia y ejecución de respaldos

Los respaldos se hacen siempre antes de tocar el stack y se guardan en /opt/backups/AAAA-MM-DD, fuera de los volúmenes de Docker. Se conservan siete días y se copia la carpeta del día al equipo del DevOps con scp.

Respaldo lógico de base de datos

```
mkdir -p /opt/backups/$(date +%F) && cd /opt/backups/$(date +%F)
for s in iam recognition lexicon analytics profile; do
  docker compose -f /opt/traduce/backend/docker-compose.yml exec -T postgres-$s
pg_dump -U postgres -Fc $s > $s.dump
done
docker compose -f /opt/traduce/backend/docker-compose.yml exec -T mongo mongodump --
archive --gzip -u signa -p "$MONGO_ROOT_PASSWORD" --authenticationDatabase admin >
mongo.archive.gz
```

Respaldo de volúmenes persistentes de Docker

```
docker run --rm -v backend_mongo-data:/data -v /opt/backups/$(date +%F):/out alpine
tar czf /out/mongo-data.tgz -C /data .
docker run --rm -v backend_rabbitmq-data:/data -v /opt/backups/$(date +%F):/out
alpine tar czf /out/rabbitmq-data.tgz -C /data .
```

Para PostgreSQL se prefiere el dump lógico del paso anterior, porque copiar los archivos de datos con el motor encendido puede dejarlos inconsistentes.

Respaldo de configuraciones de stacks y variables anteriores

- Copiar docker-compose.yml y .env a la carpeta del respaldo con permisos 600.
- Exportar el stack desde Portainer (Stacks > backend > Editor) y guardar el texto.
- Anotar el commit desplegado actualmente con «git -C /opt/traduce/backend rev-parse --short HEAD».

Gestión de variables de entorno y secretos

El archivo .env vive solo en el VPS, con permisos 600 y propietario el usuario de despliegue. No se sube a GitHub (está en .gitignore) y no se pega en chats ni documentos. El repositorio trae .env.example con los nombres de todas las variables y sin valores.

Parámetros de conexión a base de datos

POSTGRES_PASSWORD es común a las cinco bases y cada servicio recibe DB_HOST, DB_NAME y DB_PORT desde el docker-compose.yml. MONGO_ROOT_USER y MONGO_ROOT_PASSWORD arman la cadena de conexión de MongoDB. Las contraseñas no pueden llevar los caracteres @ : / ? # porque van dentro de URL.

Claves de firmado de tokens

JWT_SECRET es obligatorio, de al menos 32 caracteres y el mismo para todos los servicios: el servicio de identidad firma y los demás verifican. Se genera con «openssl rand -hex 32». Cambiarlo cierra la sesión de todos los usuarios, así que solo se rota de forma planeada.

Límites de memoria y runtime

Los servicios corren con NODE_ENV=production desde la imagen. Para evitar que un servicio consuma toda la memoria del VPS, el stack de producción añade límites en Portainer: 512 MB para recognition-service y 256 MB para cada uno de los demás servicios, con «NODE_OPTIONS=--max-old-space-size» un 20 % por debajo del límite. Las variables JVM -

Xmx y ASPNETCORE_ENVIRONMENT del formato no aplican, porque no hay servicios en Java ni en .NET.

Protocolo de comunicación y ventana de mantenimiento

- Un día antes: aviso en el grupo del equipo y al instructor con la hora de la ventana.
- Al iniciar: mensaje «inicia despliegue v1.0.0»; desde ese momento nadie hace merge a develop ni a main.
- Durante: el DevOps informa cada paso terminado; QA avisa cualquier fallo de inmediato.
- Al cerrar: mensaje con el resultado (desplegado o revertido) y el enlace a las evidencias.

Procedimiento de despliegue paso a paso (Runbook)

Verificación y acceso a la plataforma base (Portainer CE)

1. Entrar al VPS con la llave SSH y abrir el túnel hacia Portainer:

```
ssh -L 9443:127.0.0.1:9443 usuario@179.236.77.215
```

2. Si Portainer no está instalado, crearlo una sola vez:

```
docker volume create portainer_data
docker run -d --name portainer --restart=always -p 127.0.0.1:9443:9443 -v
/var/run/docker.sock:/var/run/docker.sock -v portainer_data:/data
portainer/portainer-ce:lts
```

3. Abrir https://localhost:9443 en el equipo local, iniciar sesión con la cuenta de administrador y comprobar que el entorno «local» aparece conectado.

Despliegue y migración de base de datos

Ejecución de scripts DDL/DML o herramientas de migración

El proyecto no usa Flyway ni EF Core. Cada servicio trae sus scripts SQL numerados en services/servicio/db, y el docker-compose.yml los monta en /docker-entrypoint-initdb.d, de

modo que PostgreSQL los ejecuta en orden la primera vez que se crea el volumen. Los cambios sobre una base que ya tiene datos se aplican con Liquibase, cuyos changelogs están en la misma carpeta:

```
cd /opt/traduce/backend/services/iam-service/db
cp liquibase.properties.example liquibase.properties   # completar la contraseña
liquibase update
```

En esta versión las bases ya existían en el VPS, por lo que solo se ejecuta Liquibase si «liquibase status» muestra cambios pendientes.

Verificación de integridad de esquemas, índices y usuarios

```
docker compose exec postgres-iam psql -U postgres -d iam -c "\dt"
docker compose exec postgres-iam psql -U postgres -d iam -c "SELECT name FROM
roles;"
docker compose exec postgres-lexicon psql -U postgres -d lexicon -c "SELECT count(*)
FROM sign_lexicon;"
docker compose exec postgres-lexicon psql -U postgres -d lexicon -c "\di"
```

Se acepta cuando aparecen las tablas de cada servicio, existen los roles USER y ADMIN y el alfabeto tiene sus 27 letras.

Despliegue y orquestación del stack de aplicación

Carga del archivo docker-compose.yml en Portainer

1. Actualizar el código en el VPS: «git -C /opt/traduce/backend fetch && git -C /opt/traduce/backend checkout 028d4ab».
2. En Portainer, ir a Stacks > Add stack, nombre «backend», opción Repository: URL https://github.com/mauro-1508/backendTS, referencia refs/heads/develop y ruta docker-compose.yml. Si el stack ya existe, abrirlo y usar Pull and redeploy.

Inyección de variables de entorno del stack

En la sección Environment variables del stack se cargan JWT_SECRET, POSTGRES_PASSWORD, MONGO_ROOT_USER, MONGO_ROOT_PASSWORD, RABBITMQ_USER, RABBITMQ_PASSWORD, EMAIL_USER, EMAIL_PASS y

CORS_ORIGINS. Portainer las guarda cifradas en su volumen; no se escriben en el compose. Se puede usar «Load variables from .env file» con el .env del VPS.

Despliegue de servicios y actualización de imágenes de contenedor

Se pulsa Deploy the stack con la opción de reconstruir imágenes. El equivalente por consola es:

```
cd /opt/traduce/backend
docker compose pull
docker compose up -d --build
docker compose ps
```

Los servicios esperan a que su base de datos y RabbitMQ estén sanos antes de arrancar. El paso termina cuando los doce contenedores están en estado running y los de base de datos en healthy.

Configuración de enrutamiento, dominio y certificados SSL/TLS

En esta versión se publica el gateway directamente en el puerto 8080 sin dominio. Para el siguiente despliegue:

1. Registrar el dominio y crear un registro A hacia 179.236.77.215.
2. Instalar Nginx y Certbot y obtener el certificado: «sudo certbot --nginx -d api.dominio».
3. Configurar «location /api/ { proxy_pass http://127.0.0.1:8080; }» y redirigir el puerto 80 al 443.
4. Publicar el gateway solo en 127.0.0.1:8080 y cerrar el 8080 en el firewall.
5. Cambiar la URL del gateway en la aplicación a https y quitar el permiso de tráfico sin cifrar de Android.

Plan de verificación, pruebas y evidencias (Smoke tests)

Matriz de enlaces y endpoints de verificación de servicios

Tabla 8
Endpoints de verificación

N.º	Servicio	Enlace o comando	Resultado esperado
1	Panel Portainer	https://localhost:9443 (por túnel SSH)	Stack «backend» con 12 contenedores running
2	Interfaz de la aplicación	APK instalado o Expo apuntando a http://179.236.77.215:8080/api	Pantalla de inicio y alfabeto cargados
3	Salud del backend	http://179.236.77.215:8080/health	200 OK con los cinco servicios en «up»
4	Documentación de la API	Contratos OpenAPI en docs/07-api/contracts del repositorio de documentación	Un contrato por servicio
5	Conectividad TCP a bases de datos	En el VPS: «docker compose exec postgres-iam pg_isready -U postgres»	accepting connections

Acceso al panel administrativo

Con el túnel SSH abierto, Portainer debe mostrar el stack sin contenedores detenidos ni reiniciándose en bucle.

Interfaz web pública

La interfaz se verifica en el APK, que es el canal publicado en esta versión. La versión web se valida en local con «npx expo start --web» contra el gateway del VPS, mientras se configura el dominio.

Endpoints de salud del backend

El gateway expone /health, que consulta el /health de cada servicio y responde «ok» si todos están arriba o «degraded» indicando cuál falla. No hay /actuator/health porque no se usa Spring Boot.

Documentación interactiva de la API

Los contratos OpenAPI 3 de cada servicio están en el repositorio de documentación y se pueden abrir en editor.swagger.io. El backend no publica Swagger UI en producción para no exponer la superficie de la API.

Conectividad TCP directa a motor de base de datos

Las bases no aceptan conexiones desde internet. Para revisarlas desde un equipo del equipo de trabajo se usa un túnel: «ssh -L 5433:127.0.0.1:5433 usuario@179.236.77.215» y luego un cliente como DBeaver contra localhost:5433.

Pruebas de humo funcionales

Tabla 9
Pruebas de humo

N.º	Prueba	Pasos	Resultado esperado
1	Registro	POST /api/auth/register con un correo de prueba	201 y correo con código de verificación
2	Autenticación	POST /api/auth/login con la cuenta verificada	200 con token JWT que incluye los roles
3	Token inválido	GET /api/users/me con un token alterado	401
4	Lectura	GET /api/lexicon/alphabet	200 con las 27 letras y sus modelos 3D
5	Lectura protegida	GET /api/translations/history con token	200 con el historial del usuario
6	Escritura	Traducir una seña desde la aplicación	La traducción aparece en el historial tras cerrar y abrir la aplicación
7	Evento entre servicios	Revisar «Mis logros» después de la prueba 6	Logro «Primera traducción» desbloqueado
8	Autorización	GET /api/analytics/summary con rol USER	403

Flujo de autenticación y emisión de tokens

Pruebas 1 a 3. Se usa siempre una cuenta de prueba creada para la ventana; no se usan cuentas de usuarios reales.

Pruebas de lectura y consulta de datos

Pruebas 4 y 5. Comprueban que el gateway enruta a cada servicio y que las bases tienen los datos esperados.

Pruebas de escritura y persistencia transaccional

Pruebas 6 y 7. La traducción se guarda en la base de reconocimiento y publica un evento que consumen el perfil y la analítica; si el logro aparece, la cadena completa funcionó.

Inspección de logs y monitoreo de recursos

Revisión de trazas de inicio sin excepciones críticas

```
docker compose logs --since 30m | grep -iE "error|fatal|unhandled"
docker compose logs --since 30m iam-service | grep -i mailer
```

Se acepta si no hay errores no controlados y cada servicio muestra su mensaje de arranque con el puerto.

Monitoreo de consumo de CPU y memoria RAM en contenedores

```
docker stats --no-stream
```

Ningún contenedor debe superar el 80 % de su límite de memoria en reposo, ni mantener la CPU por encima del 50 % sin tráfico.

Registro de evidencias visuales

Las capturas se guardan en la carpeta capturas junto a este documento con el nombre indicado. QA las toma al final de la ventana y las adjunta a la versión en PDF.

Tabla 10
Evidencias requeridas

N.º	Evidencia	Archivo	Estado al 15/10/2026
5.4.1	Portainer con los contenedores en estado running	07-02-portainer.png	Pendiente de captura
5.4.2	Visor de logs de los servicios del backend	07-03-logs.png	Pendiente de captura
5.4.3	Respuesta HTTP 200 del health check	07-04-health.png	Verificado: responde 200; profile-service aparece «down»
5.4.4	Interfaz de la aplicación con la consola de DevTools limpia	07-05-web-devtools.png	Pendiente de captura
5.4.5	Migración o esquema verificado en la base de datos	07-06-migracion.png	Pendiente de captura

Captura del panel de Portainer con contenedores en estado running

Vista Stacks > backend, donde se vea la columna State de los doce contenedores.

Captura del visor de logs de los servicios backend

Vista Logs del contenedor iam-service y del api-gateway con las últimas líneas de

arranque.

Captura de respuesta HTTP 200 de los health checks

Navegador o terminal con la respuesta de http://179.236.77.215:8080/health. En la verificación del 15 de octubre de 2026 el gateway respondió 200 con estado «degraded»: identidad, reconocimiento, léxico y analítica estaban arriba y profile-service no respondía. Antes de cerrar la ventana se revisa su log, se reinicia el contenedor y se repite la captura hasta obtener «ok».

Captura de interfaz web React con consola DevTools limpia

Aplicación abierta en el navegador con F12 en la pestaña Console, sin errores en rojo, después de iniciar sesión y abrir el alfabeto.

Captura de registro de migración exitosa en base de datos

Salida de «liquibase status» o de la consulta «\dt» del paso de verificación de esquemas.

Plan de reversión

- Criterio: si una prueba de humo crítica (1, 2, 4 o 6) falla y no se corrige en 20 minutos, se revierte.
- En Portainer, volver a desplegar el stack con la referencia del commit anterior anotado en los respaldos, o por consola con «git checkout» a ese commit y «docker compose up -d --build».
- Si hubo cambios de esquema, restaurar cada base con «pg_restore -U postgres -d servicio --clean servicio.dump» y MongoDB con «mongorestore --archive --gzip».
- Repetir las pruebas de humo sobre la versión anterior y registrar el incidente en el documento de acciones correctivas.

Conclusiones

El despliegue sobre el VPS con contenedores permite subir una versión completa del backend en menos de una hora y volver a la anterior con los respaldos del mismo día. La verificación inicial dejó dos tareas claras para cerrar la versión: recuperar profile-service, que aparece caído en el health check, y poner HTTPS con dominio propio para dejar de exponer el gateway por HTTP.

Referencias

Docker Inc. (2026). *Docker Compose documentation*. https://docs.docker.com/compose/

Let's Encrypt. (2026). *Getting started*. https://letsencrypt.org/getting-started/

Liquibase. (2026). *Liquibase documentation*. https://docs.liquibase.com/

Portainer.io. (2026). *Portainer CE documentation*. https://docs.portainer.io/

PostgreSQL Global Development Group. (2026). *pg_dump*. https://www.postgresql.org/docs/16/app-pgdump.html

Sommerville, I. (2016). *Software engineering* (10.ª ed.). Pearson.